

정규 표현식

□ 정규 표현식이란 무엇인가?

- 문자열에서 특정 패턴을 찾고, 검사하고, 바꾸기 위한 문법
- 실제 사용하는 예
 - 이메일, 비밀번호, 전화번호 등 형식 검사
 - 전화번호 추출 및 마스킹
 - 금지어 필터링
 - 파일 확장자 판별 등

□ 자바스크립트에서 정규 표현식 기본 형태

- 리터럴 방식
`const regex = /abc/;`
- 생성자 방식
`const regex = new RegExp("abc");`
- 플래그 사용
`/abc/g` → 전역 검색, `/abc/i` → 대소문자 무시

□ 정규식에서 자주 사용되는 기호

기호	의미	예시	설명
.	모든 문자 1개	<code>/a.c/</code>	"abc", "axc" 등
^	문자열 시작	<code>/^abc/</code>	"abc로 시작"
\$	문자열 끝	<code>/xyz\$/</code>	"xyz로 끝남"
[]	문자셋	<code>/[aeiou]/</code>	모음 중 하나
[^]	제외 문자셋	<code>/[^0-9]/</code>	숫자 제외
\d	숫자	<code>/\d+/</code>	숫자 하나 이상
\w	알파벳+숫자+_	<code>/\w+/</code>	단어처럼 생긴 것
\s	공백 문자 (space, tab, 줄바꿈)	<code>/\s+/</code>	연속된 공백 문자에 일치
+	1회 이상 반복	<code>/a+/</code>	"a", "aa", "aaa"
*	0회 이상 반복	<code>/a*/</code>	"", "a", "aa"
?	0 or 1	<code>/a?b/</code>	"b", "ab"

□ 정규식 관련 함수

목적	함수	설명
일치 여부 확인	<code>regex.test(str)</code>	true / false
위치 확인	<code>str.search(regex)</code>	인덱스 반환
첫 일치 값 추출	<code>str.match(regex)</code>	배열 또는 null
모든 일치 값 추출	<code>str.match(/.../g) / str.matchAll(regex)</code>	배열
그룹 추출	<code>str.match(...)</code> / <code>regex.exec(...)</code>	<code>result[1]</code> , <code>groups.name</code>
치환	<code>str.replace(regex, replacement)</code>	값 변경 가능

- `test()` – 일치 여부 (true/false 반환)

```
const regex = /cat/;
console.log(regex.test("I have a cat")); // true
console.log(regex.test("I have a dog")); // false
```

- `search()` – 일치 위치 반환 (없으면 -1)

```
console.log("hello world".search(/world/)); // 6
console.log("hello world".search(/cat/)); // -1
```

- `match()` – 일치한 내용 반환

```
console.log("abc123".match(/\d+/)); // [ '123', index: 3, input: 'abc123', groups: undefined ]
console.log("hello".match(/\d+/)); // null
```

- `match(/.../g)` – 모든 일치 항목 배열로 반환

```
console.log("a1 b22 c333".match(/\d+/g)); // ["1", "22", "333"]
```

- `matchAll()` – 모든 일치 항목 배열로 반환

```
for(const match of "a1 b22 c333".matchAll(/\d+/g)) {
  console.log(match); // [ '1', index: 1, input: 'a1 b22 c333', groups: undefined ]
                        // [ '22', index: 4, input: 'a1 b22 c333', groups: undefined ]
                        // [ '333', index: 8, input: 'a1 b22 c333', groups: undefined ]
}
```

- `replace()` – 일치한 부분 치환

```
console.log("apple banana".replace(/banana/, "grape")); // "apple grape"
```

- `replace()` + 그룹 활용

```
const phone = "010-1234-5678";
console.log(phone.replace(/(\d{3})-(\d{4})-(\d{4})/, "$1-****-$3")); // "010-****-5678"
```

□ 그룹 (괄호 ())

- 문자열 일부를 묶어서 추출하거나 반복 적용할 때 사용

```
const result = "apple:123".match(/(\w+):(\d+)/);
console.log(result[0]); // apple:123
console.log(result[1]); // apple
console.log(result[2]); // 123
```

- 명명된 그룹도 가능

```
const regex = /(<user>\w+):(<id>\d+)/;
console.log(regex.exec("kim:123").groups.user); // kim
```

□ 치환 예제 – replace()

- 전화번호 가운데 마스킹

```
const phone = "010-1234-5678";
console.log(phone.replace(/(?<=010-)\d{4}/, "****")); // 출력: 010-****-5678
```

- 이메일 앞글자만 남기고 마스킹

```
const email = "nodejs@example.com";
console.log(email.replace(/^(?<=)\w+/, "$1****")); // 출력: n****@example.com
```

□ 전방 탐색 / 후방 탐색

구분	문법	의미	예시
긍정 전방 탐색	(?=...)	뒤에 ...가 있는 경우	<code>\w+(?=@)/</code>
부정 전방 탐색	(?!...)	뒤에 ...가 없는 경우	<code>/foo(?!bar)/</code>
긍정 후방 탐색	(?<=...)	앞에 ...가 있는 경우	<code>/(?<=@)\w+/</code>
부정 후방 탐색	(?<!...)	앞에 ...가 없는 경우	<code>/(?<!\\$)\d+/</code>

[실습]

주민등록번호 확인

조건 1) 앞자리 숫자는 6자리, 뒷자리 숫자는 7자리로 작성되어야 한다.

조건 2) 뒷자리의 시작숫자는 1 ~ 4 이어야 한다.

○ jumin.js

```
const text = '791111-5234567';
const regex = /\d{6}-\d{7}$/;
console.log(regex.test(text));
```

[실습]

은행 계좌번호 형식 검사하기

조건 1) 숫자 3자리-숫자2자리-숫자6자리

조건 2) 첫번째 3자리 숫자의 시작은 반드시 1 이어야 한다.

조건 3) 두번째 2자리 숫자의 시작은 반드시 1 이어야 한다.

○ bank.js

```
const text = '123-12-123456';
const regex = /^1\d{2}-1\d{1}-\d{6}$/;
console.log(regex.test(text));
```

[실습]

문자열의 내용 중 이름과 전화번호 찾아내기

○ name-tel.js

```
const text = '사용자 정보, 이름: 꼬렘, 전화번호 : 1234, 이메일 : seorab@naver.com';
const regex = /이름\s*:\s*([가-힣]+).+전화번호\s*:\s*(\d+)/g;
const result = regex.exec(text);
console.log(`이름: ${result[1]}\n전화번호: ${result[2]}`);
```

[실습]

문자열의 내용 중 이메일 찾아내기

○ email.js

```
const text = '사용자 정보, 이름: 꼬렘, 전화번호 : 1234, 이메일 : seorab@naver.com';
const regex = /이메일\s*:\s*([a-z.@]+)/;
const result = regex.exec(text);
console.log(result[1]);
```

[실습]

특정 문자열의 등장 횟수 알아내기 - "harry potter"

○ harry-potter.js

```
const response = await fetch('http://ggoreb.com/quiz/harry_potter.txt');
const text = await response.text();
const regex = /harry potter/ig;
const result = text.match(regex);
console.log(result.length);
```

[실습]

특정 문자열의 등장 횟수 알아내기 - "김첨지"

○ lucky-day.js

```
const response = await fetch('http://ggoreb.com/quiz/운수좋은날.txt');
const text = await response.text();
const regex = /김\s*첨\s*지/g;
const result = text.match(regex);
console.log(result.length);
```

[실습]

형식에 맞는 모든 검색어 알아내기

○ search.js

```
const text = '#서울, #강남, #맛집, #주꾸미';
const regex = /#([가-힣]+)/g;
const result = [...text.matchAll(regex)].map(m => m[1]);
console.log(result);
```

연습문제

1. 숨어있는 문자 찾기 (special.js)

- HTML 코드 중 주석 속에 숨어있는 알파벳을 모두 찾아내기

```
<!--  
find rare characters in the mess below:  
-->
```

```
<!--  
%%$@_$_^__#)^&!+_]*@&^}@[@%]()%+$&[( _@%+%$*^@$^!+]*&_#)_*}{}}!_]${%}@[_@#_^{*  
@##&{#&{&}*%([{{([*]@[@&]++!*{!)}{ %+{))}][!^})+$]#*+^((@^@)$[*$&^{$!@#$$)!@(&  
+^!{%_&@^!}$$_{)}$#)!(@!)(^)!*^&!$%_&&}&_#&@{)}{+)%*{&*%*&@%$+]*_ (#!*){%&@++  
!_)^&&%#+)})!@!)&^}*#!_$([$!$)#*^}$+&#[{*}{((#)$)[[$$$()_#)!@^@_&%^*!){*^_$_$  
]@}#%[%!^[_]) +@&}{@*!(@$%$^)}[_!]{*}#}#___}!)(@_{{(*#%!%+* )^+#{}$+_]}%!*#*!^_  
)@)$%$^_{%!@(&{!}$$_{)*!^&{)*#(!)@)!*{^&[&$#0)*@#0_@^_#*!@_#}+[^&!@*}^){%%{&#0  
@%{&{+{#^@{)%_$_{+]}$]^^{#(*)%}@$(#[_&]#%#}{_*({()}$%[!]}#@@&_)([*]}$)&$^{^}@(%  
(%[!%]})*$}{*@)}{+@(%@*&$)*^*}*}&$[*]}%]+*}^!}*$^_()#$^]++@_) {&+((##+(&+){)$  
%&&#($[+##*%$})_!+{[_])%++)$#))]]$]@]@($+{&%&+!!!@_]++)^*@$(@#${})#}{%}#+{(@  
#_+{[]$[]!{($%$%^)(*_@+)$#$_@$)](}{}$(*%+!}#+)$%$}+#@*&^{##}+@(%[*@_]{(^]  
+_*[@+[$!@%$+{&_#^(( [&[] [&@#+_]&&]}^*&$&#)_^$@$((%) }+{ }$#+{+^}&[#_+${#}#{  
(@[@%[]]$%+*#$+[%(*!$+@$@&+$$_$#!_&&&{***+)}][ ]#^!%#&$*)$!%}*&#}##(^_%^{+}&&  
}]^#^{}@]&$)*_[]$]{_+})^_]))(^&)(!*[!&]{[()]( *}$&$_@$)$#%{[$_]!^[]}}*+  
*_^_+)^)(%{ }&[]!$&&+}&[{ }^+#$]@)^&*%{ }&{ }%*%$*&[]&{$&*@{ }$*_[]%{ }#}%)*@  
$_^#%$!{ }#^}$%@^+{($!^($%)]+&)+@$[$]*&)*%!_!!+@&^*{ }%#&{ }$!(*^*@)@)}[( $!)]))  
])(&_#*]#*%#&#*#&]@{ }_]:%<_<@&)}{+<_<)}{ }#^<_<@<)}{ }$<_<_<#*%]}{ }#&]@&_<_<1%  
-->
```

- 기본 코드

```
const response = await fetch('http://ggoreb.com/quiz/special.html');
```

```
const data = await response.text();
```

```
const text = // 코드 작성
```

```
const regex = // 코드 작성
```

```
const result = // 코드 작성
```

```
console.log(result);
```

- 문자를 찾은 후 URL을 정상적으로 입력하여 접속한 모습

http://ggoreb.com/quiz/[찾은 값 입력].html

Good!

연습문제

2. 동일한 패턴 문자 찾기 (pattern.js)

- HTML 코드 중 주석 속에 숨어있는 문자 중 조건에 맞는 문자만 찾아내기

조건 : 앞과 뒤에 대문자 3개로만 둘러싸인 소문자 1개

ex) aUSEhABDa (O), xxTYNiPERu (O), EHGRiDKJa (X), HGRiXCRZ (X)

```
<!--
find rare characters in the mess below:
-->

<!--
kAewtloYgcFQaJNhHVgXxDiQmzjfcPYbzxIWVcQsmUbCunkfxZWdZjUZMiGqhRRiUvGmYmvnJIHEmbT
MUKLECKdCthezSYBpIEIRnZugFAxDRtQPpyeCBgBfaRVvvguRXLvkAdLOeCKxsDUvBBCwdpMMWmuELeG
ENihrcPLhujoBqPRDPvfzCwadMMbkmkzCCzoTPfbRlzbQMbImxTxNniNoCufprWXxgHZpIdkoLCrHJq
vYuyJFCZtqXLhWiYzOXeglkzhVJIWmeUySGuFVmlTCyMshQtvZpPwulbOHNoBauwvuJYcmqznOBgByPw
TDQheAbsaMLjTmAOKmNsLziVMenFxQdATQIjltwtYCHyeMwQTNxbBLXWZnGmDqHhXnLHfeyvzxMhSXzd
BEBaxeapGQPttvqRvxHPEOUtIsttPDeeuGFgmDkKQcEYjuSuiGROGfYpzkQgvcCDBKrcYwHFlvPzDMEk
MyuPxvGtgSvWgrybKOnbEGhqHUXHhnyjFwSftfaiWtAOMBZEScsOSumwPssjCPIlLsPIGffDLpZzMKz
jarrjufhgxdrzywWosrbIPrasvRUpZLaUbtDHGZQtvZovHeVSTBHpitDIUIjVvWrwvhpVzeWVYhMPs
kMVcdeHzFZxTWocGvaKhhcnzRSbWslEhpeNfJaRjLwWCvKfTLhuVsJczIYFPCyrOJxOPkXhVuCqCUgE
luwLBCmqPwDvUPuBRrJZhFEXHXSbvIjqJVVFEGRUWRSHPeKUJCpMplsrVMuCMdTZPcAezRnyRTJkYyXk
oLEmjtaCsKmNOKNHygTixMKNasiFidWNbeZYYHCGMtGjITSIzRMjzOAMuhmYwincvBydQMDPaZcIRsYU
SeEnkclzGopkBozDQfXrQqHjRvIAQsiJPCsrnUawvyskbTAnjctFftReVrIBFBKiLSFGmrLsrcnZKfOU
wnCGYaMNKNhadSGMXwBaefDrMXoNeJsUaSGIwgttAqovosuhyBWwqQLkVKnRseXaaPwrMtdWjGiVXPvd
sxbXzJwjxAITPpWOGONPwcSbHFZjBIZKEBUECMLUKQRvVwgludKQKnJmIUoUCaAHITKVKclpMelTJ
AYIVsrjBTLsWuyYwCJuJaewQsrzwXJNLafIzrZXBBEOCTtItNptaJkriIEhufpNysjJpeWbWcFqdEsiG
-->
```

- 기본 코드

```
const response = await fetch('http://ggoreb.com/quiz/pattern.html');
```

```
const data = await response.text();
```

```
const text = // 코드 작성
```

```
const regex = // 코드 작성
```

```
const result = // 코드 작성
```

```
console.log(result);
```

- 문자를 찾은 후 URL을 정상적으로 입력하여 접속한 모습

http://ggoreb.com/quiz/[찾은 값 입력].html

Good!